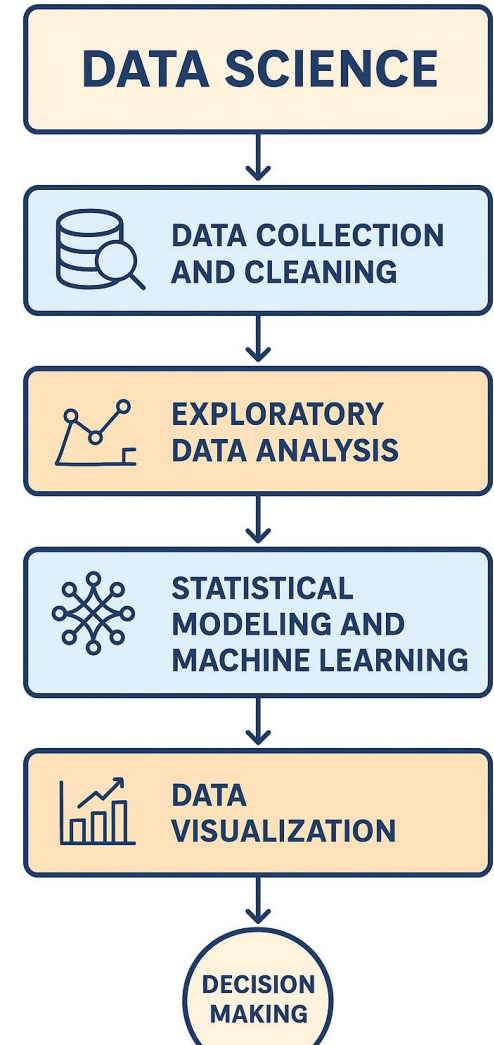


Chapter 2:

Data Preparation

Outlines

- 1 Data Collection
- 2 Data Cleaning
- 3 Data normalization
- 4 Data Reduction



Data Collection

Getting data into Python and into the right formats

Data Collection

- Collecting data is the process of assembling all the data you need from relevant sources.
- Data resides in many data sources and has vastly different formats such as images, text, video, audio, data from social networks, online websites or other data sources.

Data Collection

There are two different methods of collecting data:

- Primary Data Collection: data collected first-hand by the researcher for a specific purpose.
 - Surveys & Questionnaires
 - Interviews
 - Experiments
 - Direct Observations
- Secondary Data Collection: data collected by someone else
 - Government reports
 - Online databases
 - Research articles & journals
 - Company records
 - Social media analytics

Web crawler

Web crawlers are also referred to as *spiders* or *robots*, the resources on the Web are dispensed widely across globally distributed sites.

Web crawler

- Two primary types of data available on the Web that are used by mining algorithms:
 1. Web content information: the actual web documents and links created by users.
 - Document Data: extracted directly from Web pages (text, images, multimedia).
 - Linkage Data: the web can be viewed as a massive graph, where:
 - Nodes = Web pages
 - Edges = hyperlinks connecting those pages
 2. Web usage data: patterns of user activity captured through Web applications.
 - Web transactions, ratings, and user feedback: interactions by users such as purchases, likes, reviews.
 - Web logs: browsing behavior stored in server logs, which record user visits, clicks, and navigation paths.

Web crawler applications

The applications on the Web are either content- or usage-centric:

- Content-centric applications: search, clustering, and classification:
 - Data mining applications.
 - Web crawling and resource discovery.
 - Web search.
 - Web linkage mining.
- Usage-centric applications: The user activity on the Web is mined to make inferences:
 - Recommender systems.
 - Web log analysis.

A Basic Crawler Algorithm

Algorithm *BasicCrawler*(Seed URLs: S , Selection Algorithm: $\mathcal{A}$)
begin
 $FrontierList = S$;
 repeat
 Use algorithm $\mathcal{A}$ to select URL $X \in FrontierSet$;
 $FrontierList = FrontierList - \{X\}$;
 Fetch URL X and add to repository;
 Add all relevant URLs in fetched document X to
 end of $FrontierList$;
 until termination criterion;
end

Web Scraping

- Web Scraping is the process of automatically extracting data from websites

```
<html>
  <head>
    <title>A web page</title>
  </head>
  <body>
    <p id="author">Joel Grus</p>
    <p id="subject">Data Science</p>
  </body>
</html>
```

Web Scraping

- To get data out of HTML, use BeautifulSoup library which builds a tree out of the various elements on a web page and provides a simple interface for accessing them.

```
from bs4 import BeautifulSoup
import requests
```

```
# I put the relevant HTML file on GitHub. In order to fit
# the URL in the book I had to split it across two lines.
# Recall that whitespace-separated strings get concatenated.
```

```
url = ("https://raw.githubusercontent.com/"
       "joelgrus/data/master/getting-data.html")
html = requests.get(url).text
soup = BeautifulSoup(html, 'html5lib')
```

Exercise

Crawl admission data from <https://ptithcm.edu.vn/>

- 2025
- Other years

Using APIs

- Many websites and web services provide application programming interfaces (APIs), which allow you to explicitly request data in a structured format.
- If you need data from a specific site, look for a developers or API section of the site for details, and try searching the Web for “python __ api” to find a library.

Exercise: Get data by Twitter APIs

Data cleaning

Data cleaning is important because the data collection process often contains errors. There are several causes of missing values or errors during data collection.

Important aspects of data cleaning

- Handling Missing Values:
 - Many values in the data may be unknown and missing.
 - The process of estimating missing values is also called imputation.
- Handling Incorrect Values:
 - In cases where information comes from multiple sources, it may not be consistent.
 - Eliminating contradictions is part of the analysis process.
 - Data points that do not conform to the distribution of the remaining data are often noisy as outliers

Handling Missing Values

1. Deletion Methods
2. Single Imputation Methods
3. Multiple Imputation Methods
4. Model-Based / Advanced Methods

Handling Missing Values

1. Deletion Methods

- Listwise Deletion: remove rows with missing values.
- Pairwise Deletion: use available data pairs (common in correlation analysis).

ID	Math	Science	English
1	80	70	90
2	90	NaN	85
3	NaN	85	88
4	75	80	NaN

Handling Missing Values

2. Single Imputation Methods

- Mean / Median / Mode Imputation
- Hot Deck Imputation (use a similar record's value).
- Cold Deck Imputation (use external reference values).
- Regression Imputation (predict missing value using regression).
- Last Observation Carried Forward (LOCF) (time series).

Handling Missing Values

2. Single Imputation Methods

- Mean / Median / Mode Imputation

Example:

A dataset of student test scores:

[85, 90, NaN, 88, 92]

Mean = $(85+90+88+92)/4 = 88.75$

Replace missing value (NaN) with 88.75 (mean)

If using median, replace with 89.

If categorical (e.g., colors [red, blue, NaN, red]), replace with mode = red.

Handling Missing Values

2. Single Imputation Methods

- Hot Deck Imputation (use a similar record's value).

Example:

Name	Gender	Age
Anna	Female	25
Bob	Male	30
Emma	Female	NaN

Emma is Female → choose a similar record (Anna) → impute Age = 25.

Handling Missing Values

2. Single Imputation Methods

- Cold Deck Imputation (use external reference values).

Example: Missing salary values in a dataset.

Instead of calculating from the dataset, use an **official average salary** from a government report to fill missing entries.

Handling Missing Values

2. Single Imputation Methods

- Regression Imputation (predict missing value using regression).

Example: A dataset of height and weight:

Height (cm)	Weight (kg)
170	65
180	75
175	NaN

Build regression model: $\text{Weight} \approx a * \text{Height} + b$.

Predict missing Weight for 175 cm: ≈ 70 kg.

Handling Missing Values

2. Single Imputation Methods

- Last Observation Carried Forward (LOCF) (time series).

Idea: Common in time series → fill missing with last available value.

Example:

Stock prices: [100, 102, NaN, NaN, 105]

LOCF → [100, 102, 102, 102, 105].

Handling Missing Values

3. Multiple Imputation Methods

- MICE (Multiple Imputation by Chained Equations): iteratively imputes missing values for each variable using regression models based on the other variables.
- Bayesian Multiple Imputation

ID	Math	Science	English
1	80	70	90
2	90	NaN	85
3	NaN	85	88
4	75	80	NaN

Handling Missing Values

4. Model-Based / Advanced Methods

- Maximum Likelihood with Expectation–Maximization Algorithm:
 - Step 1: Identify the likelihood of observed data (with some missing values)
 - Step 2: Repeat until convergence.
 - E-step: Estimate missing values given current parameter estimates.
 - M-step: Update parameters using completed-data likelihood.
- Machine Learning Models
 - k-Nearest Neighbors
 - Random Forest
 - ...

Example

```
import pandas as pd
df = pd.read_csv('titanic.csv')
df.info() # check the data information using
df.duplicated() # check the duplicate rows
# Categorical columns
cat_col = [col for col in df.columns if df[col].dtype == 'object']
# Numerical columns
num_col = [col for col in df.columns if df[col].dtype != 'object']
# Check the total number of Unique Values in the Categorical Columns
df[cat_col].nunique()
```

Example

```
# Dropping Observations with missing values  
df.dropna(subset=[...], axis=0, inplace=True)
```

```
# Imputing the missing values from past observations  
df.fillna(...)
```

To check the outliers, we generally use a box plot that shows a variable's median, quartiles, and potential outliers

```
import matplotlib.pyplot as plt  
plt.boxplot(df3['Age'], vert=False)  
plt.ylabel('Variable')  
plt.xlabel('Age')  
plt.title('Box Plot')  
plt.show()
```

Codes: data_clean.py

Handling Incorrect Values

Some of the main methods used to handle incorrect or inconsistent values are as follows:

- Detect data inconsistencies
- Domain knowledge
- Data-centric approach

Handling Incorrect Values

Some of the main methods used to handle incorrect or inconsistent values are as follows:

- Detect data inconsistencies : violate logical, statistical, or format rules.
 - Range checks: e.g., Age = 1500 (impossible).
 - Format checks: e.g., phone number has letters.
 - Cross-field validation: e.g., EndDate < StartDate is inconsistent.
 - Statistical detection: outlier detection methods (e.g., if height = 5m).

Handling Incorrect Values

Some of the main methods used to handle incorrect or inconsistent values are as follows:

- Domain knowledge: Use domain knowledge to detect and correct suspicious values.
 - Apply rules derived from the real-world context.
 - Consult experts to decide how to correct outliers/inconsistencies.

Handling Incorrect Values

Some of the main methods used to handle incorrect or inconsistent values are as follows:

- Data-centric approach: Focus on improving the quality of data itself
 - Cleaning pipelines: build systematic processes to detect & correct errors.
 - Deduplication: remove duplicate records.
 - Standardization: enforce consistent units (e.g., kg vs. lbs, USD vs. EUR).
 - Consistency checks across systems

Data normalization

Data normalization

- Normalization involves transforming the values of features to a specific range.
- Common normalization methods:
 - Min-Max normalization
 - Standardization (or Z-Score) normalization

Min-max normalization

Let A be a numeric attribute with n observed values, $v_1, v_2, \dots, v_n$.

Min-max normalization performs a linear transformation on the original data.

- Suppose that $\min A$ and $\max A$ are the minimum and maximum values of an attribute, A .
- Min-max normalization maps a value, v_i of A to v'_i in the range $[\text{new_min}_A, \text{new_max}_A]$ by computing

$$v'_i = \frac{v_i - \min_A}{\max_A - \min_A} (\text{new_max}_A - \text{new_min}_A) + \text{new_min}_A$$

Min-max normalization

Example:

Suppose that the attribute *income* ranges from \$12,000 to \$98,000

By min max normalization, a value of \$73,600 for income is transformed to 0.716 in the range [0.0, 1.0]

$$v'_i = \frac{v_i - \min_A}{\max_A - \min_A} (\text{new_max}_A - \text{new_min}_A) + \text{new_min}_A$$

Min-max normalization

```
from sklearn.preprocessing import MinMaxScaler  
data = [[-1, 2], [-0.5, 6], [0, 10], [1, 18]]  
scaler = MinMaxScaler()  
print(scaler.fit(data))  
print(scaler.data_max_)  
print(scaler.transform(data))  
print(scaler.transform([[2, 2]]))
```

Z-score normalization

Let A be a numeric attribute with n observed values, $v_1, v_2, \dots, v_n$.

- z-score normalization (or zero-mean normalization), the values for an attribute, A , are normalized based on the mean and standard deviation of A .
- A value, v_i , of A is normalized to v'_i by computing

$$v'_i = \frac{v_i - \bar{A}}{\sigma_A},$$

- z-score normalization is useful when the actual minimum and maximum of attribute A are unknown, or when there are outliers that dominate the min-max normalization.

Z-score normalization

Example:

Suppose that the mean and standard deviation of the values for the attribute *income* are \$54,000 and \$16,000, respectively.

With z-score normalization, a value of \$73,600 for income is transformed to 1.225.

$$v'_i = \frac{v_i - \bar{A}}{\sigma_A},$$

Z-score normalization

```
from sklearn.preprocessing import StandardScaler  
data = [[0, 0], [1, 1], [3, 2], [3, 3]]  
scaler = StandardScaler()  
print(scaler.fit(data))  
print(scaler.mean_)  
print(scaler.transform(data))  
print(scaler.transform([[2, 2]]))
```

Data reduction

Data reduction

- Data reduction is a technique to reduce the size of a dataset while still preserving the most important information.
- It is mostly useful when your data set is large or it has a large number of dimensions and you want to find a small subset that captures most of the variation.
- 3 strategies:
 - numerosity reduction: replace the original data volume by alternative, smaller forms of data representation
 - sampling
 - dimensionality reduction: reduce the number of random variables or attributes under consideration
 - feature subset selection
 - wavelet transforms
 - principal components analysis
 - data compression.

Data reduction

- Numerosity reduction

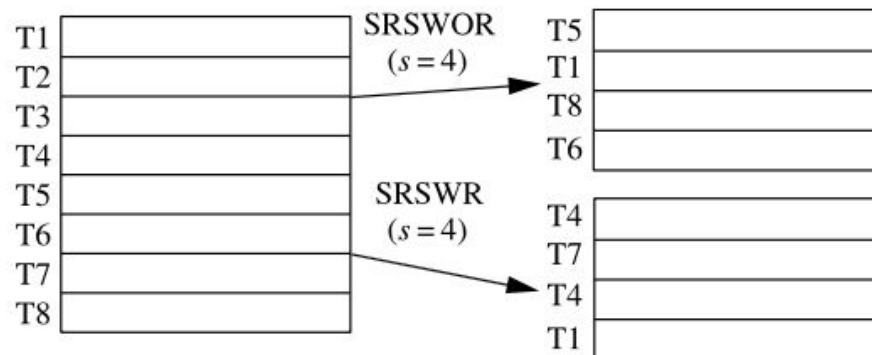
Sampling:

Suppose that a large data set, D , contains N tuples, the most common ways that we could sample D for data reduction

- Simple random sample without replacement (SRSWOR) of size s : drawing s of the N tuples from D ($s < N$)
- Simple random sample with replacement (SRSWR) of size s : it is similar to SRSWOR, except that after a tuple is drawn, it is placed back in D so that it may be drawn again

Data reduction

- Sampling



Data reduction

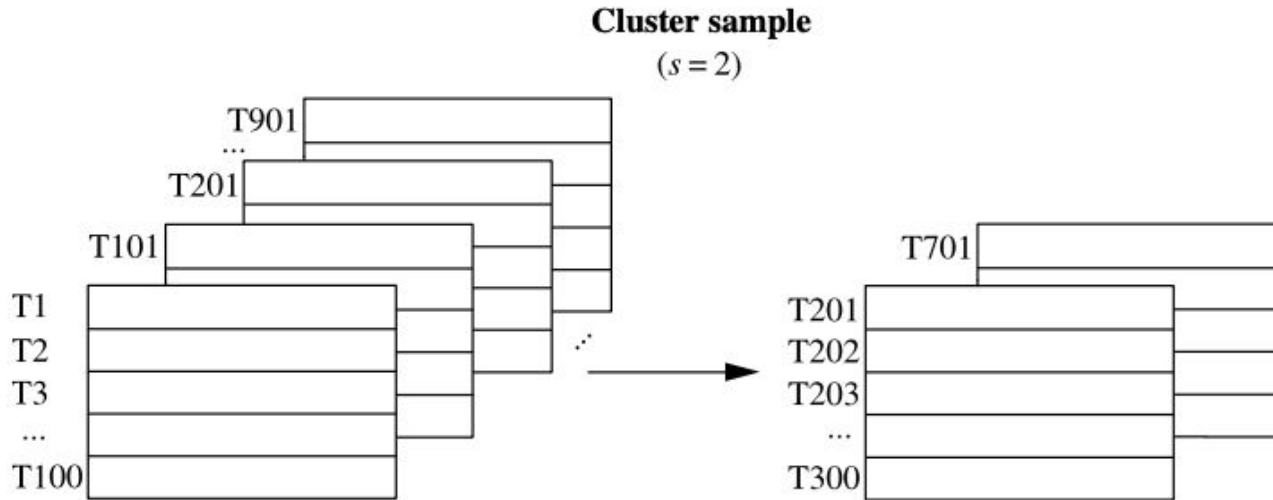
- **Numerosity reduction**

Sampling: Suppose that a large data set, D , contains N tuples, the most common ways that we could sample D for data reduction

- Cluster sample: If the tuples in D are grouped into M mutually disjoint “clusters,” then an SRS of s clusters can be obtained, where $s < M$.

Data reduction

- Numerosity reduction: Sampling



Data reduction

- **Numerosity reduction**

Sampling: selecting a subset of the data to work with, rather than using the entire dataset. Suppose that a large data set, D , contains N tuples, the most common ways that we could sample D for data reduction

- Stratified sample: If D is divided into mutually disjoint parts called strata, a stratified sample of D is generated by obtaining an SRS at each stratum.
 - For example, a stratified sample may be obtained from customer data, where a stratum is created for each customer age group.

Data reduction

- Numerosity reduction: Sampling

Stratified sample
(according to *age*)

T38	youth
T256	youth
T307	youth
T391	youth
T96	middle_aged
T117	middle_aged
T138	middle_aged
T263	middle_aged
T290	middle_aged
T308	middle_aged
T326	middle_aged
T387	middle_aged
T69	senior
T284	senior

T38	youth
T391	youth
T117	middle_aged
T138	middle_aged
T290	middle_aged
T326	middle_aged
T69	senior

Data reduction

Dimensionality reduction: reduce the number of random variables or attributes under consideration.

- **Feature subset selection:** reduce the data set size by removing irrelevant or redundant attributes (or dimensions)

Greedy (heuristic) methods for attribute subset selection.

1. Stepwise forward selection:

- The procedure starts with an empty set of attributes as the reduced set.
- The best of the original attributes is determined and added to the reduced set.

2. Stepwise backward elimination:

- The procedure starts with the full set of attributes. At each step, it removes the worst attribute remaining in the set.

Data reduction

Forward selection	Backward elimination
Initial attribute set: $\{A_1, A_2, A_3, A_4, A_5, A_6\}$ Initial reduced set: $\{\}$ $\Rightarrow \{A_1\}$ $\Rightarrow \{A_1, A_4\}$ $\Rightarrow$ Reduced attribute set: $\{A_1, A_4, A_6\}$	Initial attribute set: $\{A_1, A_2, A_3, A_4, A_5, A_6\}$ $\Rightarrow \{A_1, A_3, A_4, A_5, A_6\}$ $\Rightarrow \{A_1, A_4, A_5, A_6\}$ $\Rightarrow$ Reduced attribute set: $\{A_1, A_4, A_6\}$

Data reduction

Dimensionality reduction

- **Feature subset selection:** reduces the data set size by removing irrelevant or redundant attributes (or dimensions)

Greedy (heuristic) methods for attribute subset selection.

3. Combination of forward selection and backward elimination:

The stepwise forward selection and backward elimination methods can be combined so that, at each step, the procedure selects the best attribute and removes the worst from among the remaining attributes.

Data reduction

Dimensionality reduction

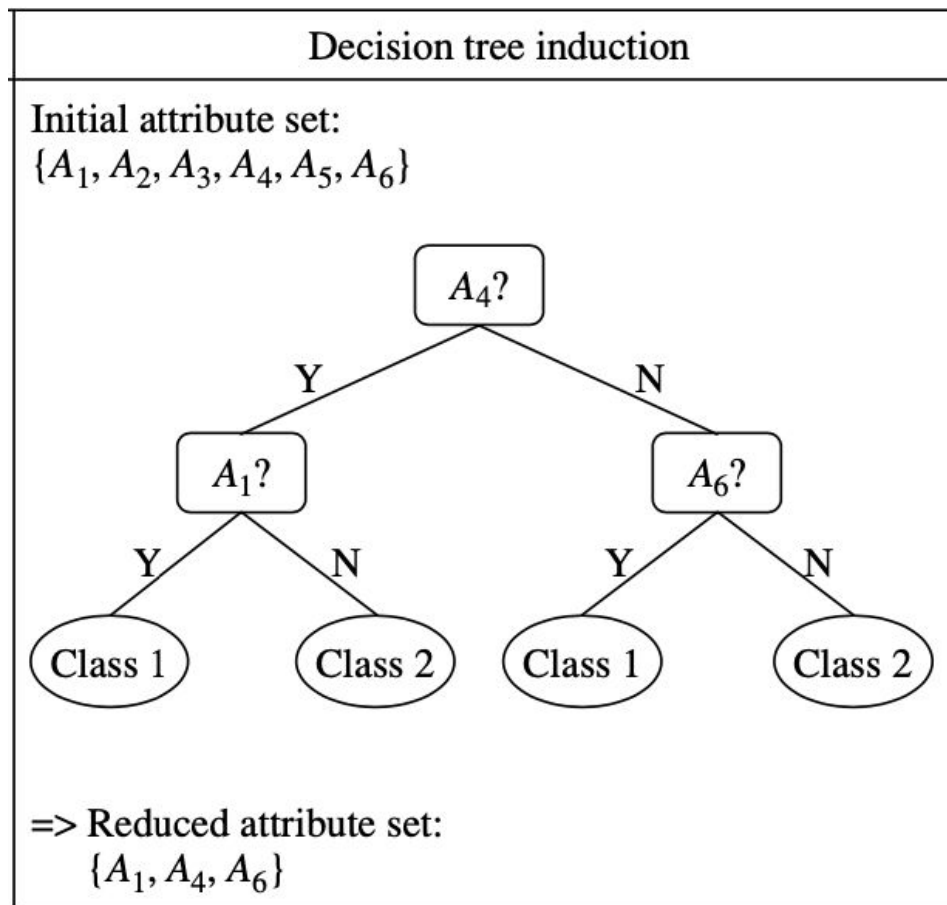
- **Feature subset selection:** reduces the data set size by removing irrelevant or redundant attributes (or dimensions)

Greedy (heuristic) methods for attribute subset selection.

4. A tree is constructed from the given data.

All attributes that do not appear in the tree are assumed to be irrelevant. The set of attributes appearing in the tree form the reduced subset of attributes.

Data reduction



Data reduction

- **Dimensionality reduction**

- Principal Components Analysis**

Suppose that the data to be reduced consist of tuples or data vectors described by n attributes or dimensions.

- PCA searches for k n -dimensional orthogonal vectors that can best be used to represent the data, where $k \leq n$.
- Algorithm:
 - Standardize the data (important if variables are on different scales).
 - Compute the covariance matrix (or correlation matrix).
 - Find eigenvalues and eigenvectors:
 - Eigenvectors = directions of principal components.
 - Eigenvalues = amount of variance explained by each component.
 - Sort components by eigenvalues (largest first).
 - Select top- k components to form the reduced dataset.

Data reduction

```
from sklearn import datasets, decomposition
```

```
iris = datasets.load_iris()
```

```
X = iris.data
```

```
pca = decomposition.PCA(n_components=3)
```

```
pca.fit(X)
```

```
X = pca.transform(X)
```